A Comparative Study of Full Apps and Lite Apps for Android

Anonymous Author(s)

ABSTRACT

App developers aim to create apps that cater to the needs of different types of users. This development approach, also known as the "one-size-fits-all" strategy, involves combining various functionalities into one app. However, this approach has drawbacks, such as lower conversion rates, slower download speed, larger attack surfaces, and lower update rates. To address these issues, developers have created "lite" versions to attract new users and enhance the user experience. Despite this, there has been no study conducted to examine the relationship between lite and full apps. To address this gap, we present a comparative study of lite apps, exploring the similarities and differences between lite and full apps from various perspectives. Our findings indicate that most existing lite apps fail to fulfill their intended goals (e.g., smaller in size, faster to download, and using less data). Our study also reveals the potential security risks associated with lite apps.

CCS CONCEPTS

• **Software and its engineering**; • **General and reference** → **Empirical studies**;

KEYWORDS

Empirical Study, Lite Apps, Android, Static Code Analysis

1 INTRODUCTION

Over these years, mobile devices and applications have become a ubiquitous part of our daily lives. According to AppBrain statistics [5], there are over 2.68 million Android apps available on the Google Play Store. To meet the needs of different types of users, developers are continually striving to build apps with rich features. This often leads to apps that are packed with various features and libraries to support different application binary interfaces (ABIs) for different platforms [22, 34]. In this context, a feature is defined as a specific functionality in an app that satisfies a particular requirement. For example, the ability to "pay a bill" in a bank app can be considered a feature as it satisfies the transaction needs of a bank app user.

Combining all features into an app, also known as the "one-size-fits-all" strategy, can offer convenience and suitability for different types of users and devices. However, such a "one-size-fits-all" strategy can bring various potential defects and risks to the target systems, including lower conversion rates, slower download speeds, larger attack surface, and lower update rates [9]. To combat the software bloat in Android apps, two types of solutions are proposed [34]: first, *pruning-based debloating approach* removes unwanted features from the app to build a debloated app [23, 34]. The debloated app has fewer features than the full app. For example, developers can leverage the pruning-based delobating approach to remove an out-of-date module from an app. There are two representative works in this field: RedDroid [23] performs debloating in two aspects: (1) code redundant debloating: RedDroid regards "dead code" as redundant and removes them; and (2) ABI redundant debloating: RedDroid considers ABIs that are not suitable for the target CPU architecture as redundant and removes them from the app. For example, if the CPU architecture is ARMv7, then ABIs for ARMv5, X86, MIPS(64), ARMv8, X86, X86 64 are pruned. Different from RedDroid, XDebloat allows developers to prune unwanted features from an app [34]. Specifically, developers can first annotate features they intend to prune. Then, XDebloat prunes them to build a debloated app. Second, *module-based debloating approach* allows developers to organize apps into modules. Thus, developers can download and install modules in an on-demand manner. One representative framework is the App Bundle Framework [18] offered by Google. With the app bundle framework, an app bundle is organized by one base module and zero or more feature modules. When an app bundle is downloaded, its base module is downloaded and its feature modules are downloaded on demand.

Motivation. Nevertheless, automating the process of debloating an app presents its fair share of challenges. Developers and researchers face several challenges when debloating apps. **C1** The uniqueness of Android apps: Unlike traditional software, features in an app can vary due to the unique nature of the Android system. For example, Android apps interact with users through UI elements and have more entry points than traditional software. **C2** Defining features in apps: Compared to traditional software, more factors need to be considered when defining features in an app, such as permissions, hardware, and UI. **C3** UI-related issues: Pruning apps is different from pruning traditional software as UI-related issues must be taken into account during the pruning.

Goal of this study. In this study, we aim to gain insights from the practices of real-world app developers by examining how they create "lite" versions of their apps. Normally, a lite app is defined as an app that has a limited set of features in comparison to its full version. These lite apps are developed by official app developers and can be considered as a form of debloating in comparison to the full version of the app. Understanding the methods and considerations used by developers in creating lite apps can (1) assist researchers in developing effective debloating approaches and tools; and (2) assist developers in creating their own lite apps by refactoring their legacy apps.

Contribution. In summary, we make the following contributions in this paper:

- In this paper, we conduct a *first* comparative study of lite apps and their counterpart full apps. We collect 260 lite-full app pairs from Google Play;
- We conduct a systematic quantitative and qualitative analysis on the app pairs from various aspects, including non-code similarity, code, similarity, permission, privacy policy, resource consistency, performance improvement, features included and excluded in lite apps, and potential security concerns in lite apps; and
- The results of this study shed a light on understanding the existing practices of building lite apps, and show the limitations and potential drawbacks of existing lite apps.

2 MOTIVATION EXAMPLE

Investigating lite apps can offer valuable insights for researchers and developers. Lite apps are designed to be a compact version of their full app counterparts, and comprehending the design choices and functional disparities between the two can assist in the development of debloating techniques and tools. This study utilizes the example of LinkedIn and its LinkedIn Lite app to illustrate the design of lite apps and the variations between them and full apps.

2.1 Meta Comparison

Table 1: LinkedIn App vs. LinkedIn Lite App

	LinkedIn	LinkedIn Lite	Similarity
Apk Size	80.1 MB	8.1 MB	/
# Permissions	41	27	100%
# Activity	25	14	50%
# Content Provider	9	3	0%
# Service	23	14	42.85%
# Broadcast Receiver	17	14	50%
# Methods	56163	7066	28.93%
# Resources files	17031	187	1.37%

LinkedIn is an online service focused on employment, offering job seekers opportunities and allowing companies to post job listings. To gain an understanding of the two types of apps, we compare the APK files of LinkedIn and its Lite app. Table 1 presents the differences between the two apps in 8 aspects including Apk size, number of permissions, number of `Activity`, number of `Content Provider`, number of `Service`, number of `Broadcast Receiver`, number of methods, and number of resource files. Furthermore, we determine the similarity between the two apps using Eq. 1, as shown in Table 1. As displayed, all permissions used in the Lite app are included in the LinkedIn app, with 100% of permissions, 50% of `Activities`, 42.85% of `Services`, 50% of `Broadcast Receivers`, 28.93% of methods, and 1.3% of resource files present in both apps.

2.2 Functionality Comparison

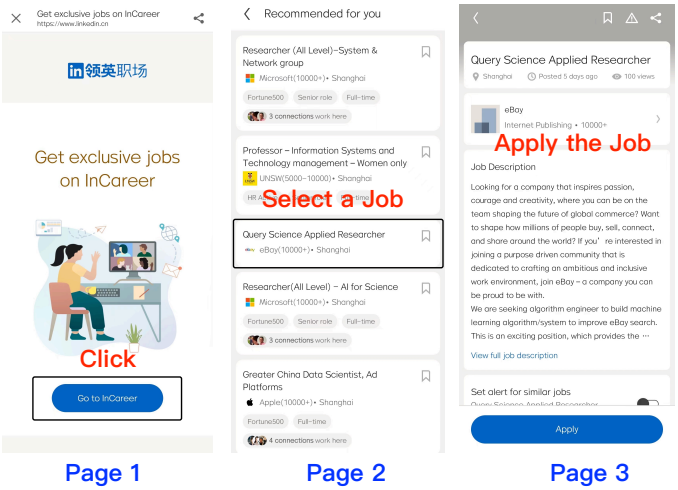

Figure 1: UI and Basic Workflow of LinkedIn Lite App

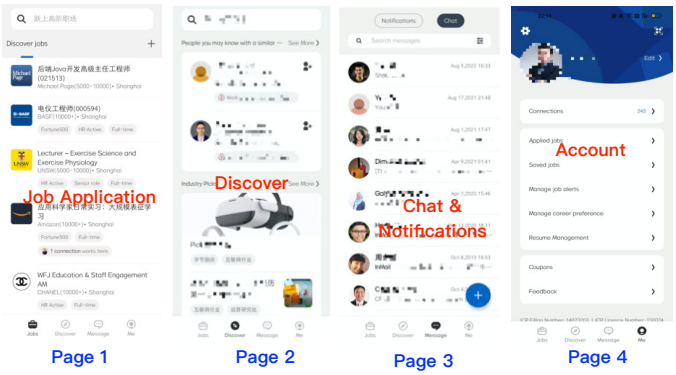

Figure 2: UI and Basic Workflow of LinkedIn App

We perform a functionality comparison between LinkedIn app and its lite app. Fig. 1 shows the main UI and basic workflow of the lite app. As shown, after logging in, users can apply for recommended jobs using pre-saved resumes. In contrast, Fig. 2 illustrates the main user interface and functions of the LinkedIn app. It includes four main features: **job application**, **discover**, **messaging (chat and notifications)**, and **account management**. Notably, the job application feature is the same in both the lite and full versions, with the only difference being that users can select recommended jobs in the full version, while job searching is not allowed in the lite version. The **discover** feature (second tab) offers suggestions for "people you may know" and "news from companies". The **messaging** feature (third tab) allows users to send messages to their connections and receive notifications. Lastly, the **account management** feature (last tab) allows users to upload resumes, manage their account, set preferences, view saved jobs, and view applied jobs among other options.

Table 2: Functionality Comparison between LinkedIn and Lite

Features	LinkedIn	LinkedIn Lite
Login	✓	✓
Logout	✓	✗
Job Apply	✓	✓
Job View	✓	✓(Recommended job only)
Job Search	✓	✗
Job Share/Save	✓	✓
Discover	✓	✗
Message(Chat,Notification)	✓	✗
Account Management	✓	✗
Preferences	✓	✗
Resume Management	✓	✗

The LinkedIn and Lite apps offer different features to cater to the needs of different users. The LinkedIn offers a wide range of features such as job search, messaging, and account management, while the LinkedIn Lite only offers the core feature of job application. As shown in Table 2, the lite app only contains the key feature of job application, compared to the full app. Developers have intentionally limited the features in the lite app to provide a lightweight version for users who only need the core functionality. Additionally, the UI consistency between the two apps is maintained to effectively

reduce the learning costs for users and allow them to apply for recommended jobs in a timely manner.

3 EXPERIMENT AND EVALUATION

3.1 Research Questions

In this paper, we intend to answer the following research questions (RQ).

• **RQ1 (Non-code Similarity):** How similar are the disassembled permissions/resource files between lite apps and full apps?
• **RQ2 (Code Similarity):** To what extent is the code of lite apps similar to their full apps?
• **RQ3 (App Descriptions):** Whether lite apps and their corresponding full apps can have dramatically difference app descriptions?
• **RQ4 (Permission, Privacy Policy Consistency):** How do lite apps' claimed permissions, privacy policies stay consistent?
• **RQ5 (Performance Improvement with Lite Apps)**: Whether lite apps bring performance improvement comparing to full apps?
• **RQ6 (Shared, Included and Excluded Features in Lite Apps)**: What are the functionalities are shared, included and excluded in lite apps?
• **RQ7 (Potential Secure Concerns in Lite Apps)**: What are the potential secure concerns (permission, privacy) in lite apps?

3.2 Data Collection

To our knowledge, there is no publicly available dataset for lite and full app pairs for our study. Therefore, we created our own dataset through the following steps: 1) randomly selecting 400,000 apps from Google Play [19], 2) grouping the apps by developers/-companies to ensure that the lite and full app pairs are from the same developer/company, and 3) grouping the lite and full apps by comparing their app descriptions and using a similarity rate of 85%. This resulted in 299 app pairs, with 39 of them being paid apps. After removing these paid apps, we were left with 260 app pairs.

3.3 RQ1: Non-Code Similarity

Motivation. In this RQ, we intend to evaluate the non-code similarity between lite apps and full apps.

Methodology. We first investigate the similarity of lite app and full app pairs in terms of non-code assets. Specifically, we compare the following aspects: (1) declared permissions; (2) resource files: images, videos, audios, libraries, and other resource files.

(1) Declared permissions: App permission system is a front-line mechanism to protect the privacy of Android users. If an app intend to use certain sensitive data or access to hardware resources, its have to claim certain permissions from the Android OS. If and only if such permission is granted, the app can access to certain data or hardware service. We leverage Android Asset Packaging Tool (AAPT) [3] to obtain the declared permissions in an app; and

(2) Resource files: An app can leverage different types of multimedia resources, such as images, videos, audio. In general, we compare file names and the hash value of two files with the same name in the lite version and full version.

Results. As shown in Fig. 3, 50% of lite apps reuse 87.5% permissions from their full apps; 75% apps reuse more than 58.3% permissions.

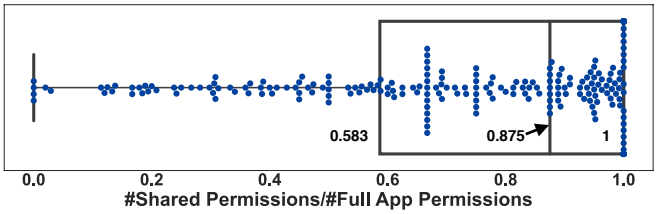

Figure 3: Boxplot of Permission Reuse Ratio

★ The high reuse ratio of permission benefits in (1) consistency: by reusing the same permissions, the behavior and functionality of the lite apps can be consistent with the full apps, which can improve the user experience; and (2) simplicity: the development process can be simplified, as the developer does not need to consider which permissions are needed for the lite app version. However, the high reuse ratio can also bring some drawbacks, such as, privacy concerns and security concerns. We discuss this issue in RQ 3.

We also compare which kinds of permissions are removed when building lite apps. The top 10 removed permissions are: `gms.permission.AD_ID` (44,16.9%), `vending.BILLING` (38,14.6%), `FOREGROUND_SERVICE` (35,13.5%), `RECEIVE_BOOT_COMPLETED` (34,13%), `ACCESS_COARSE_LOCATION` (29,11.1%), `SYSTEM_ALERT_WINDOW` (29,11.1%), `CAMERA` (28,10.7%), `POST_NOTIFICATIONS` (28,10.7%), `READ_PHONE_STATE` (27,10.3%), and `ACCESS_WIFI_STATE` (26,10%).

After inspecting the lite apps that these permissions are pruned, we find the potential concerns for removing these permissions are:
(1) **Performance concerns and data consumption**: The top-1 removed permission is `gms.permission.AD_ID`, which is related to Google Ad. The top-2 removed permission is used for in-app purchase. By removing data consumption operations and services, developers saves computing resources in the lite apps;
(2) **Service (notification)**: Developers also prefer to set services (e.g., foreground service, geo-service) in the full app as these services need long-running operations in the background. Such long-running operations can be inappropriate to lite apps, as this may be contrary to the original intention of lite apps; and
(3) **Network Requirement**: Developers also pay attention to the network requirements for lite app users. We find that developers use these permission to detect whether the target devices are connected to WIFI. For example, the full app `com.amazon.klite` checks the WIFI status to determine when to upload the crash reports to the remote server.

For the resource files **reuse** perspective, we use the `Similarity` metric defined in SimiDroid. In SimiDroid, the extracted key/value mapping pairs (map1,map2) are then compared based on four metrics: (1) **identical**, which indicates an exact match between compared key/value pairs; (2) **similar**, which indicates the compared pairs have the same key but different value; (3) **new**, which indicates key exists only in map1 (i.e., the component/method only exists in the lite version); and (4) **deleted**, which indicates the key only exists in map2 (i.e., the component/method only exists in the full version). The similarity score is calculated as

$$similarity = max\left\{\frac{identical}{total - new}, \frac{identical}{total - deleted}\right\}, \quad (1)$$

where, total=identical+similar+new+deleted.

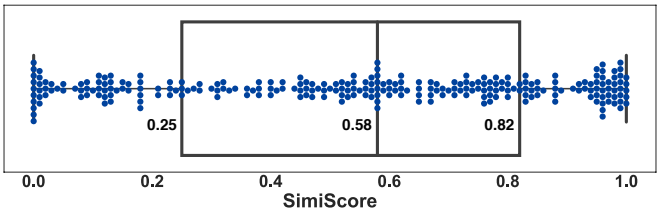

Figure 4: Boxplot of Resource SimiScore

As a result, the metric `similarity` (i.e., SimiScore) can be used to compute the similarity between two apps. On average, 54.27% resource files in lite apps are reused from full apps. As the boxplot of SimiScore shown in Fig. 4, the medium value is 0.58, and the interquartile range is 0.25 to 0.82. To understand the motivation, we manually inspect 3 app pairs for each category. We find that it is mainly due to the design choices and concerns for lite apps. Different developers may have different design choices and concerns for building lite apps. Some developers only intend to cover a few feature in the lite apps and leave most features in the full apps. While some developers intend to refine and redesign some features in the lite apps. As a result, the reuse rates for apps can vary dramatically.

RQ1: Non-Code Similarity

- The potential concerns for removing these permissions in the lite apps are (1) performance concerns and data consumption; (2) service (notification); and (3) network Requirement;
- On average, the apps in the following categories have a reuse rate over 50%: Communication, Social, Game, Map, Tools, Videos, Education, Entertainment, Life, Productivity, Personalization, Traveling, Dating;

3.4 RQ2: Code Similarity

Motivation. Typically, lite apps and full apps have similar functionalities, but due to different goals for each app version, understanding the similarities and differences in code between a lite app and a full app can provide developers and researchers with a deeper understanding of the relationship between the two.

Methodology. We leverage SimiDroid [25], which compares the pairwise similarities and differences between Android apps. Specifically, SimiDroid is built upon Soot, which transforms the source code into an intermediate representation named Jimple [32]. The metric `similarity` can be used to compute the similarity between two apps in terms of code.

Table 3: Average Number of Components in Lite Apps and full Apps

	Activity	Service	Broadcast Receiver	Content Provider
Lite Apps	26	4	2	1
full Apps	54	6	4	2

Results. ▸ **Comparison at Component Level.** When comparing lite apps to their full app counterparts, it is clear that developers tend to reduce the number of interactions offered in lite apps, both in the foreground and background. This is evident in the average number of `Activity` (i.e., the number of UI screens) and the number of background services (i.e. `Service`), which are approximately half that of those found in full apps. This suggests that developers aim to offer lightweight features, reduce complexity, and improve performance in lite apps. Similar patterns can also be observed in the number of broadcast receivers and content providers in lite apps compared to full apps.

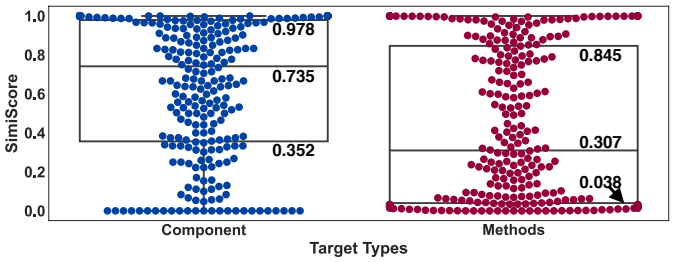

Figure 5: SimiScore of the Lite and full App Pairs

The left side of Fig. 5 illustrates that, on average, 63.9% of the components in lite apps are identical to those in their corresponding full apps. 50% of lite apps have a SimiScore higher than 0.735, indicating that 73.5% of components from the full app counterparts are reused. This highlights that a substantial number of components are transferred without modification and that lite apps often retain the same user interaction as their full app counterparts.

▸ **Comparison at Method Level.** The right-hand side of Fig. 5 illustrates the SimiScore between lite and full app pairs in terms of methods. On average, 43% of the methods in lite apps are identical to those in their corresponding full apps. 50% of lite apps have a similarity score higher than 0.307, meaning that 30.7% of methods from the full app are reused. The figure also shows that 75% of lite apps have a similarity score higher than 0.038 (i.e., 3.8% of methods), and 25% of lite apps have a similarity score of 0.845 (i.e., 84.5% of methods) are reused.

In summary, this figure suggests that most app developers only include a few features in their lite app versions, but still maintain the majority of the components.

Table 4: Detailed Simidroid Results

	Identical	Similar	New
Components	52.33%	1.46%	46.19%
Methods	38.11%	4.17%	57.70%

Furthermore, the detailed result is shown in Tab. 4. It shows a detailed view of average percentage of identical, similar, new components or methods of a lite app compared with its full app counterpart.

★ This suggests that (1) compared to lite apps, full apps typically have around twice as many additional features or components; and (2) a higher reuse rate of components compared to methods in lite apps suggests that developers strive to provide lightweight features in lite apps while still preserving a consistent user interface for users, in order to improve performance.

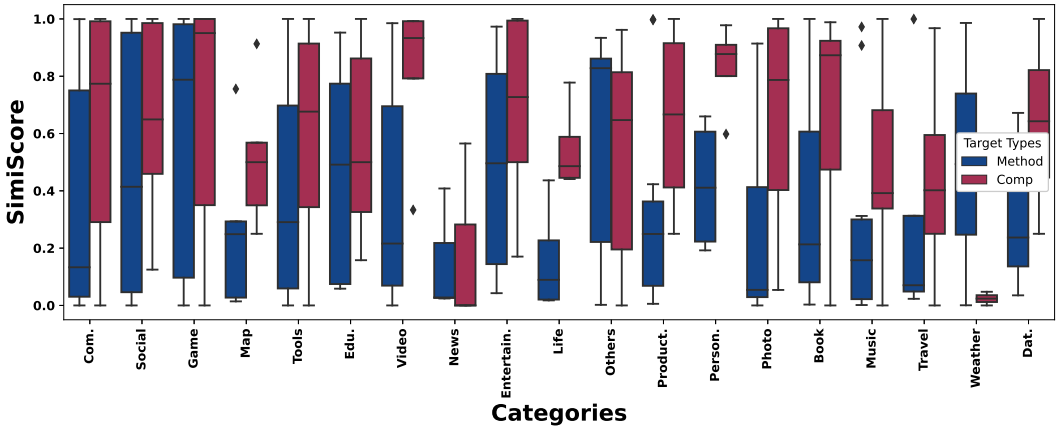

Figure 6: SimiScore of App Pairs by Category.

As illustrated in Fig. 6, the similarity scores for components and methods vary across different app categories. However, a notable exception is the *Video* and *Personalization* categories, which have a consistently high component-level SimiScore compared to other categories. After thoroughly analyzing apps in the *Video* and *Personalization* categories, we found that most user interactions and components in both lite and full versions are identical. Specifically, developers from these two categories often do not make changes to the way users interact with the app, resulting in identical user interfaces and interactions in both versions. In the case of *Video* apps, developers tend to maintain consistency in terms of user interaction, as evidenced by the identical components in both lite and full versions. In the case of *Personalization* apps, the ways of interaction are relatively simple, thus there is little difference between the lite and full versions.

RQ2: Code Similarity

- A higher reuse rate of components compared to methods in lite apps suggests that developers strive to provide lightweight features in lite apps while still preserving a consistent user interface for users, in order to improve performance;
- The component and method level similarities in all categories do not exhibit huge differences, except for the apps in *Video* and *Personalization* categories whose component-level SimiScore is significantly stable and higher than the other categories.

3.5 RQ3: App Description Similarity

Motivation. In this research question, we intend to discuss whether lite apps and their corresponding full apps can have dramatically difference app descriptions.

Methodology. For 260 app pairs, we collect their app descriptions from Google Play Store for comparison. Next, we clean the text data by removing stop words (common words like 'the', 'is', 'in', etc.), punctuation, and converting all text to lowercase. This step helps to reduce noise in the text and makes the comparison more accurate. Furthermore, we use cosine similarity to compare app descriptions.

Results. As shown in Fig. 7, the majority of lite apps and their corresponding apps share the similar app descriptions. On average,

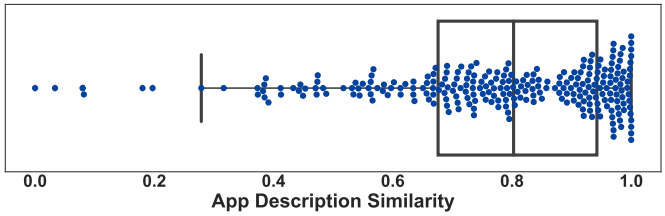

Figure 7: Boxplot of App Description Similarity

the lite apps and their corresponding full apps have 77.42% similarity in terms of app descriptions. The medium value is 80.19%, which suggests there are 50% of apps have similarity not less than 80.19%. There are 28 app pairs (out of 260) have similarity values less than 50%. Next, we further investigate the lite apps and their corresponding full apps have dramatically different descriptions. As a result, we find the following clues that may lead to dramatic differences in app descriptions.

- **Functionality and Features**: A lite version of an app generally has fewer features compared to its full version. Developers may cut out certain functions or remove some advanced features to simplify the user interface. Consequently, the app descriptions can be different to reflect these changes;
- **Target Audience**: The target audience for the lite version might be different than that of the full version. For example, a lite version may be aimed at users in emerging markets or users with older devices. Thus, the marketing language used in the description may be different to better appeal to this new audience.
- **Usage & Interaction**: The way users interact with the app may also change. For example, the lite version of a social media app might be more focused on basic functions like posting status updates or photos, while the full version may allow more advanced features like live streaming.

RQ3: App Description Similarity

For the majority of app pairs, the app descriptions are similar. However, developers may design unique app descriptions for lite apps with the following possible concerns: different features/-functionalities, different target audiences, and different usages/interaction modes.

3.6 RQ4: Permission, Privacy Policy Consistency

Motivation. Intuitively, lite apps can be created by reworking existing full apps. However, this process can be prone to errors, which can open up opportunities for attackers. In this research question, we are interested in three aspects: (1) permission consistency: we check whether the permissions claimed are actually used in the lite app; (2) privacy policy: we examine whether (a) developers have created tailored privacy policies for lite apps, and (b) whether the privacy policies in lite apps align with the permissions they request; and (3) resources: we check whether resource files are defined but not used in the lite app.

Methodology. (1) For consistency of **permissions**, we leverage the PScout[7] to find the mappings between APIs and permissions. We leverage PScout to find out between APIs and permissions for Android API 19. The result is reused to scan the code to determine the required permissions. Next, we compare the in-use permissions and the claimed permissions to find out over-claimed permissions; (2) For consistency of **privacy policy**, (a) for a pair of apps, we check whether the policy for the lite app is the same as the policy for the full app, to determine whether developers have created tailored privacy policies. (b) To check consistency between permissions and policies, we use AutoCog [30], which uses a learning-based algorithm to link the descriptions with permissions; and

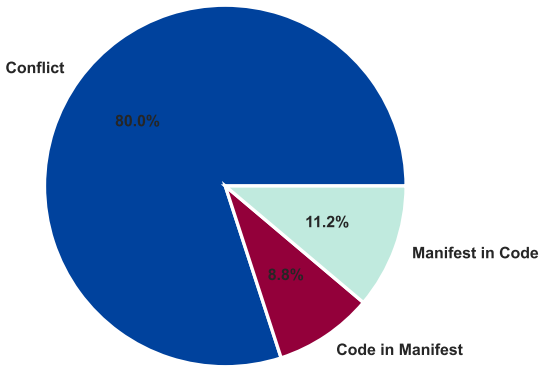

Figure 8: Results of Permission Consistency

Results. ► For consistency of **permissions**, we examine all 260 lite apps and find that none of these apps maintain permission consistency. Specifically, as shown in Fig. 8, there are 23 apps claim more permissions in AndroidManifest files than they used in code; 29 apps that require more permissions than they claim in the AndroidManifest files; and other 208 apps contain permission conflicts. That means, these 208 apps claim permissions that they donot use and use permissions but not claimed in the AndroidManifest files. Furthermore, we find that 89 (out of 208) apps' permission conflicts are rooted in third-party libraries; 122 (out of 208) apps' permission conflicts are rooted in main app bodies; and 3 apps' have conflicts in both main bodies and third-party libraries.

★ Through out inspection of these apps, we have discovered that the possible causes for these conflicts may stem from: (1) third-party libraries: conflicts may arise if libraries used by the app request additional or different permissions than those required by the app itself, (2) codebase changes: the process of reengineering a legacy app into a lite app can result in changes to the codebase, but developers may overlook updating the corresponding permissions accordingly.

► For **privacy policy** consistency, we find only 239 apps contain privacy policies. For the rest 21 apps, they do not have privacy policies published. For these 239 apps, there are only 12 apps comply with the privacy policy consistency. For the rest 227 apps, on average, each apps have 13 permissions are used but not claimed in the privacy policies, which shares a proportion of 67.5%. It means 67.5% of permissions are used but not but not claimed in the privacy policies. Furthermore, we find that there 163 apps out of 239 apps (68.2%) reuse the privacy policies from their full apps.

★ Since lite apps have distinct functionality and required permissions, to ensure proper user data protection, developers must meticulously craft new privacy policies specifically tailored to the lite app rather than simply recycling the ones from the full app version.

RQ4: Permission Privacy Policy and Resources Consistency

- For **permission** consistency, we find that there are 23 apps claim more permissions in AndroidManifest files than they used in code; 29 apps require more permission than that they claimed in the AndroidManifest files. For **privacy policy** consistency, we find that for these 239 apps, there are only 12 apps comply with the privacy policy consistency. For the rest 227 apps, on average, each apps have 13 permissions are used but not claimed in the privacy policies, which shares a proportion of 67.5%.
- While the presence of unused resources or overclaimed permissions may not always signify a security vulnerability, it is a best practice to thoroughly review and remove them to decrease potential risks.

3.7 RQ5: Performance Improvement with Lite App

Motivation. In this RQ, we aim at evaluating whether lite apps bring performance improvement comparing to full apps from aspects: (1) size comparison; (2) startup time; (3) memory consumption; (4) CPU consumption; and (5) network requests.

Methodology. Specifically, to evaluate (2) **startup time**, we leverage the official the time to initial display (TTID) metric, which measures the time it takes for an application to produce its first frame, including process initialization (if a cold start), activity creation (if cold/warm), and displaying first frame. The TTID can be obtained with `logcat`. To evaluate (3) **memory consumption** and (4) **CPU consumption**, we leverage the adb command "`adb shell dumpsys meminfo |grep <package-name>`" and "`ps -p <pid> -o TIME`", respectively. Note that, here "TIME" represents the amount of CPU time used by the process. For example, "0:01:10" represents the CPU time used by the process is 1 minute and 10 seconds. To evaluate (5) network requests, we perform the following steps: a) we use Soot's hierarchy analysis to find all the classes that are related to network communication with HTTP and HTTPS protocols, including, `java.net.URL`, `java.net.URLConnection`, `java.net.HttpURLConnection`, `javax.net.ssl.HttpsURLConnection`,

`okhttp3.Request` ,and `okhttp3.OkHttpClient`; b) next, we iterate through each method of the class and use Soot's control flow analysis to find all the method invocations of `openConnection()`, `connect()`, `sendRequest()`, `execute()` and other similar methods that are related to making network requests. The URL can be extract with Def-Use analysis; and c) last, we extract the URLs used in the invocations, and record the URLs and the corresponding classes and methods in which they are used.

Configuration. To conduct our study, we use an Android emulator with the following specifications: (1) CPU/ABI: Google Play Intel Atom (x86); (2) Android API 30; and (3) 512M of memory. We utilize the Android Monkey tool to generate pseudo-random streams of user events for 15 minutes, and then calculate the memory and CPU usage.

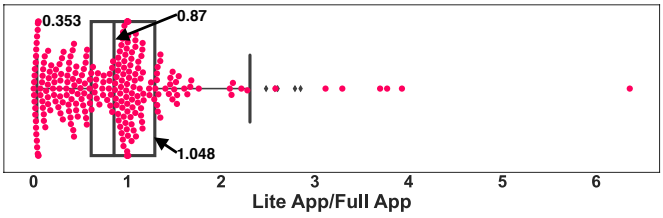

Figure 9: Size Comparison (lite/full) Boxplot

Results. ►**Size Comparison.** As illustrated in Fig. 9, on average, lite apps are 83.2% the size of their corresponding full apps. The largest proportion observed was 6.359, while the smallest was 0.001. Half of the apps have a proportion less than 0.87, with 25% having a proportion less than 0.353 and 75% having a proportion less than 1.048. Furthermore, 66.7% of lite apps are smaller in size compared to their full app counterparts.

★ Contrary to common belief, lite apps are not always smaller in size and have fewer features compared to their full version counterparts. As shown in Fig. 9, surprisingly, around 33% of lite apps have a larger size than their full version counterparts.

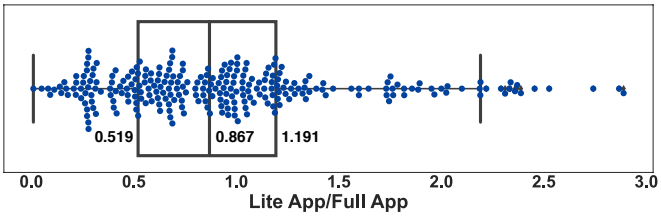

Figure 10: Startup Time Comparison (lite/full) Boxplot

►**Startup Time.** As illustrated in Fig. 10, on average, the lite apps have a startup time that is 94.6% of the full app counterparts. The maximum proportion is 2.889, and the minimum proportion is 0.009. Half of the apps have a startup time that is less than 0.867 of the full app counterparts, 25% of apps have a startup time ratio lower than 0.519, and 75% of apps have a ratio lower than 1.191. Additionally, 61.5% of lite apps have a startup time that is shorter than their full app counterparts.

►**Memory Consumption.** As seen in Fig. 11, on average, lite apps consume 0.946 times the memory of their full app counterparts.

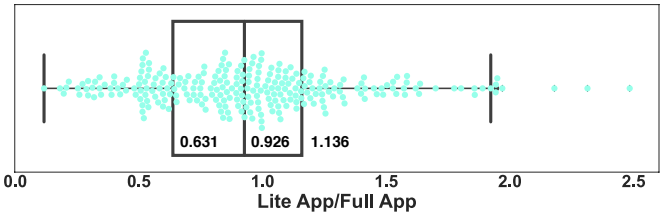

Figure 11: Memory Comparison (lite/full) Boxplot

The highest proportion observed is 2.484, while the lowest is 0.116. Half of the apps have a proportion lower than 0.926, 25% have a proportion lower than 0.631, and 75% have a proportion lower than 1.135. This suggests that the memory consumption of 60.7% of lite apps is not larger than that of their full app counterparts.

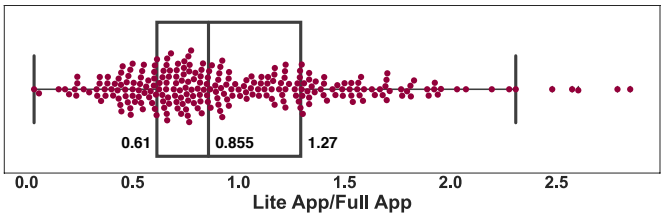

Figure 12: CPU Time Comparison (lite/full) Boxplot

►**CPU Consumption.** As shown in Fig. 12, on average, lite apps have a CPU consumption that is 98.5% of their full app counterparts. The maximum proportion observed is 2.848, while the minimum is 0.03. Half of the apps have a proportion less than 0.855, 25% of apps have a proportion less than 0.61, and 75% of apps have a proportion less than 1.27. Additionally, 60% of lite apps have a lower CPU consumption compared to their full app counterparts.

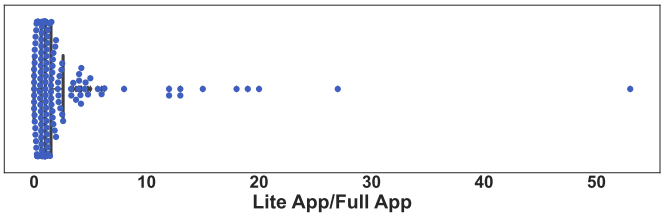

Figure 13: Network Requests Comparison(lite/full) Boxplot

►**Network Requests.** As represented in Fig. 13, The network requests proportion range from 0 to 53 with an average of 2.11. Furthermore, the medium value of network requests is 52 for lite apps and 53 for full apps. This suggests the lite apps send more network requests than full apps.

Furthermore, we find mere 24.6% of lite apps exhibit improvements across all performance metrics compared to their full app counterparts. The remaining lite apps even consume more resources than their full app versions.

RQ5: Performance Improvement with Lite apps

• The size proportion of lite apps to full apps ranges from 0.001 to 6.359, with an average of 0.832. The startup time proportion of lite apps to full apps ranges from 0.009 to 2.889, with an average of 0.946. The memory consumption proportion of lite apps to full apps ranges from 0.116 to 2.484, with an average of 0.946. The CPU consumption proportion of lite apps to full apps ranges from 0.03 to 2.848, with an average of 0.985; The network requests proportion of lite apps to full apps ranges from 0 to 53, with an average of 2.11.
• As seen in our analysis, a mere 24.6% of lite apps exhibit improvements across all performance metrics compared to their full app counterparts. The remaining lite apps even consume more resources than their full app versions.

3.8 RQ6: Shared, Included, Excluded Features in Lite Apps

Motivation. Our aim in this RQ is to understand the functional differences between lite apps and their full app counterparts. By examining which features are included and excluded in lite apps, we can gain insight into how developers decide what to include in these lightweight versions (e.g. lite apps).

Methodology. We use the Diffuse toolkit [11] to compute the differences between the apks of lite apps and their full app counterparts. This toolkit allows us to identify the specific functionalities that are included and excluded in the lite apps. To further analyze these differences, we collect all identifiers, such as method names, variables, and class names, from the apks. We then use Latent Dirichlet Allocation (LDA) to extract relevant topics from these identifiers, with each class file being considered as a document.

Results. In order to understand the feature differences between lite apps and their full app counterparts, we analyze three key areas: (1) functionality excluded in lite apps, (2) functionality included in lite apps, and (3) consistency in functionality between lite apps and full apps

►**Functionality excluded in lite apps.** There are top-5 features that are used in their full app counterparts instead of lite apps:
• *Complex data management:* In full apps, developers normally place some advanced data management features such as data syncing;
• *Audio and video processing and support:* In full apps, full apps often include advanced audio and video processing capabilities/support with third-party libraries to process, load images and videos;
• *Advanced support and frameworks for Games and Graphical User Interface:* Full apps often include some advanced support and frameworks (e.g., Unity Game Engine) for 3D animations, transitions, physics engines, and multiplayer support;
• *Support for Other Devices:* Besides, we also find that full apps may offer additional support for third-party devices such as game controllers, and wearables; and
• *Complex and Advanced Account management:* Different from lite apps, full apps often offer some advanced features for account managements, such as, multiple login options, account recovery.

►**Functionality included in lite apps.** Top-5 features/functionalities that are included in lite apps, including
• *Lightweight and effective data processing and management:* Different from full apps, lite apps normally have more effective and lightweight data processing and management. For example, most lite apps use `Parcelable`, which is an interface that allows objects to be passed between activities, services, and other components in a more efficient way comparing to Serialization;
• *Simplified concurrency and Performance Improvement:* Comparing to full apps, developers are more prone to adopt frameworks, such as Coroutines[4], to manage asynchronous code;
• *Custom Class/Resources Loading Mechanism* Lite apps are prone to use `dalvik.system.DelegateLastClassLoader` to provide a custom class loading mechanism. This gives additional benefits to lite apps, including (1) using a custom class loading mechanism can potentially improve the performance of the app by reducing the overhead of the class loading process; and (2) it allowed developers to easily add support for third-party libraries, by specifying the location of the library's class files in the classpath. The similar mechanism is used by lite apps to load resources dynamically, which can reduce the sizes of lite apps;
• *Network connection and detection:* Lite apps often include some features to detect and check network connection status in order to determine whether it is needed to turn off background data or limit the amount of data transferred; and
• *Reduced graphics and animations:* Lite apps often include reduced graphics and animations to reduce data usage.

►**Consistency in functionality between lite apps and full apps** There are several features that are top-5 commonly found in both lite apps and their full app counterparts:
• *Account management:* Both lite apps and full apps often include user account management features, such as, login, logout, and profile management;
• *Notification:* The notification features in lite apps and full apps are sent to users to inform them of new content, updates, or activities within the app;
• *Social integration:* We also find that social integration features are commonly shared by lite apps and counterpart full apps to allow users to login social account, such as Google, Twitter;
• *Advertising:* Both lite and full apps may include advertising features to monetize their apps;
• *App Performance Analytic:* Both lite and full apps leverage app performance management libraries, such as, Google Analytics [20], OKHTTP3 [29], Flurry [14], to track crashes and app usages;
★ In conclusion, two key insights emerge when designing features for lite apps: (1) Lite apps can function as a gateway for users to test and explore the app, potentially leading to an upgrade to the full version. (2) Performance optimization is crucial in the development of lite apps, as this will help to reach a wider audience.

RQ6: Shared Included and Excluded Features in Lite Apps

• **Shared features in lite and full apps:** account management, notification, social integration, advertising, and app performance analytic; **Included features in lite apps:** lightweight data processing and management, simplified concurrency, customized class/resource loading, network connection and detection, and reduced graphics and animations; **Excluded features in lite apps:** complex data management, audio/video processing, advanced support for games and GUIs, cross-platform features, and advanced account management;

3.9 RQ7: Potential Security Concerns in Lite Apps

Motivation. Lite apps may not have the same level of security as their full app counterparts. In this RQ, we focus on discussing the potential secure concerns from two aspects: (1) whether dangerous permissions are introduced to lite apps; (2) whether lite apps may have more sensitive data leaks than full apps. The first aspect is important as lite apps may introduce new permissions that can be exploited by attackers. The second aspect is important as lite apps may have different data handling practices than full apps, which could lead to sensitive data leaks.

Methodology. Dangerous permissions in Android refer to a set of permissions that can potentially put the user's data or the device at risk. There are 38 dangerous permissions declared by Android. For (1), we check whether dangerous permissions are introduced to lite apps with AAPT. For (2), we leverage FlowDroid, which is a state-of-art taint analysis tool for Android, to detect potential privacy leaks in lite apps and full apps. Specifically, we make the following configurations in FlowDroid.

Results. ►**Dangerous Permissions:** The results of our experiments indicate that out of 260 apps, 211 (81.15%) contain at least one dangerous permission. Furthermore, among these 211 apps, there are 83 lite apps (31.9%) that introduce dangerous permissions that are not present in their corresponding full apps. The top introduced dangerous permissions are: GET_ACCOUNTS (10 times), READ_EXTERNAL_STORAGE (9), COARSE_LOCATION (8), READ_PHONE_STATE (8), WRITE_EXTERNAL_STORAGE (7), POST_NOTIFICATIONS (6), FINE_LOCATION (5), WRITE_CALENDAR (5), and BACKGROUND_LOCATION (4).

It suggests a significant number of lite apps introduce dangerous permissions, which are not present in their full app counterparts, potentially putting users at risk for security and privacy breaches. Therefore, it is important for developers to be aware of the potential dangers and for users to be mindful of the permissions they grant to lite apps. Furthermore, we inspect these lite apps that introduce dangerous permissions and find the following motivations:

(1) To improve performance: developers may introduce dangerous permissions in lite apps to improve performance by accessing certain resources or data on the device. For example, developers offer complex and advanced account management support in full apps. In the lite apps, to improve performance, developers ask users to access the account through accounts Accounts Service (e.g. using a Google account), which requires the GET_ACCOUNTS permission in lite apps for user authentication. As a result, a dangerous permission (e.g.,GET_ACCOUNTS) is introduced in lite apps; and

(2) To collect more data from users: developers may desire to collect more data from users in order to improve their apps, promote their full apps, or target advertising more effectively.

►**Privacy Leaks:** Out of 520 apps (260 pairs), FlowDroid crashes on 3 apps with runtime errors. As for the rest of 257 app pairs, as represented in Fig. 14, 178 lite apps out of 257 lite apps have privacy leaks, and 165 full apps contain privacy leaks. To our surprise, 161 lite apps (62.6%) introduce new privacy leaks comparing to their corresponding full apps. On average, the lite apps have 11.4 leaks which are higher than that (8.4 leaks) in full apps. Furthermore, the medium value is 4 in lite apps, whereas that is 2 in full apps. The

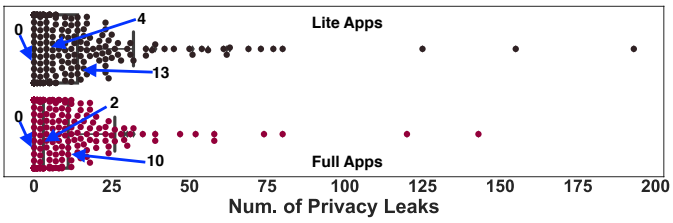

Figure 14: Number of Leaks in Apps

results indicate that lite apps offer fast and streamlined experience, particularly on devices with limited resources. However, some most lite apps have been found to potentially compromise user privacy by leaking sensitive information.

We further investigated the causes of privacy leaks in lite apps. We find the the following reasons from the samples: (1) third-party performance libraries: lite apps utilize third-party performance libraries and APIs to collect data from target users to analyze and improve their apps; (2) logging: logging libraries and APIs are used to debug, which may lead to privacy leaks that go unnoticed; and (3) implementation replacement: lite apps are designed to be lightweight, often resulting in reduced functionality, which lead to potential leaks.

RQ7: Potential Security Concerns in Lite Apps

- There are 83 lite apps (31.9%) that introduce dangerous permissions that are not present in their corresponding full apps. It suggests a significant number of lite apps introduce dangerous permissions, which are not present in their full app counterparts, potentially putting users at risk for security and privacy breaches.
- We find that 62.6% of lite apps introduce additional privacy leaks, which are not shown in the full apps.

4 DISCUSSION AND INSIGHTS

4.1 Implication and Research Directions

The findings and conclusions drawn in this study lead to several potential insights and opportunities for researchers to work on.

Implication 1: Inconsistency and Privacy Issues in Lite Apps. The driving force behind the development of lite apps is to offer improved user experience on low-end devices with limited resources such as low memory and processing power. The aim of lite apps is to be smaller in size, faster to download, and consume less data compared to traditional native apps. However, our study found that the majority of existing lite apps fall short of their intended goals. For instance, our results indicate that some dangerous permissions not present in full apps can be introduced in lite apps, and some lite apps also violate privacy policy consistency. Only 24.6% of the lite apps bring performance improvements compared to full apps. These findings suggest that developers must ensure that their lite apps deliver a seamless user experience, are efficient in their resource usage, and perform well on low-end devices.

Implication 2: (Semi-)automatic Migration from Full Apps to Lite Apps. As presented in our study, we find that approximately 50% of the non-code resources, approximately 31% of code in lite apps are directly reused from the full apps. Moreover, the number

of lite apps are still limited on app stores. One possible reason is that developers still have to build the lite apps from scratch. It can be promising to build lite versions of apps by migrating legacy apps. Thus, this is a need for building the lite versions of apps by automatic or semi-automatic migrating existing legacy apps. This also benefits the development of lite apps and boost the lite app ecosystem. Our study shed the light on building such a (semi-)automatic tool.

Our study also suggests several potential research directions for lite apps that could deepen our understanding of this technology and its potential impact:

Research Directions 1: Predicting Lite App Features. Research should be conducted to predict which functionalities are most critical for lite apps and tailored users with deep learning techniques.

Research Directions 2: Integration with Full App. Research should be conducted to offer the option to upgrade to the full app for users who need more functionality.

Research Directions 3: Improving the Data Collection Practices in Lite Apps. Research should be conducted on exploring alternative monetization models for lite apps that do not rely on data collection, such as subscriptions or in-app purchases.

4.2 Threats to Validity

The validity of the results may be impacted by the limited number of app pairs studied. The findings of this study should not be generalized to the entire app ecosystem. The method used, static analysis, may have limitations in accurately analyzing complex language features such as reflection and dynamic loading. However, these language features are not the focus of the similarity comparison, so their impact on the results is minimal. Other factors such as device and network conditions that may affect the results were not controlled for or taken into account in this study.

5 RELATED WORK

5.1 Code Clone and App Similarity

For efficiency, developers may clone the existing code into current projects. An et al.[2] studied the code clone from Stack Overflow to Android apps, and they find that code reuse from Stack Overflow may violate the license and cause a copyright issue. Fischer et al.[13] found that 15.4% of the 1.3 million Android apps they analyzed contained security-related code snippets from Stack Overflow, and 97.9% of these code snippets contained at least one unsafe code segment. Gharehyazie et al.[16] studied the cross-project code clones in Github, they build a tool called CLONE-HUNTRESS to track code clones in Github. Due to code clones or other reasons, different apps may have similarities. Frenklach et al.[15] built the app similarity graph and use the app similarity graph to detect Android malware. Gonzalez et al.[17] presented DroidKin, a tool to detect the similarity between Android apps. Zhou et al.[38] build a system called DroidMOSS to detect app similarity based on fuzzy technology.

5.2 App Analysis

There are many works on analyzing apps. Here, we only present some representative works. Arzt et al.[6] presented the FlowDroid, a static taint analysis that has high efficiency and precision for Android apps. The Pscout[7] maps Android APIs to Android permissions. A static analyzer can use the Pscout to find the permissions used by the app. Cao et al.[10] built the EdgeMiner to detect implicit control flow transitions in the Android framework automatically. Li et al.[24] presented the Violist, a string analysis framework to extract the string value from the target APIs in Android apps. The GUI analysis for Android apps mainly contributes to the understanding of apps' testing and behaviors [8, 21, 26, 27, 31, 33, 35–37]. Specifically, Rountev and Yang [31] proposed a static object reference analysis approach for GUI Objects in apps. The static object reference analysis approach employs a constraint graph to model the flow of GUI objects and the hierarchical structure of these objects. Amalfitano et al. [1] proposed the AndroidRipper, a UI driven ripper that tests Android apps via their GUI. Wang et al [35] presented a novel analysis named *input context analysis* on an app's GUI to detect whether there exist privacy leaks for user-entered data for a given app and determining whether such leakage can lead to a violation of the privacy policy claimed. Yang et al. [36, 37] proposed a static analysis approach to build the window transaction graph for Android apps. Guo et al [21] first conduct a systematical study on learning the limitations of existing app GUI fuzzers. Then, they proposed a novel GUI fuzzing approach by leveraging additional dependencies relations between UI objects. There are also some works explore automatically testing Android apps by exploring pages in the app under test [8, 26, 27, 33].

5.3 App Debloating

The bloated apps consume more resources and may have more security vulnerabilities compared to full apps. McDaniel et al.[28] explored the bloatware in smartphones. Elahi et al.[12] found an average of 172 bloated apps in the smartphone inspected; most of them can access sensitive data. To detect and remove software bloat in apps, Tang et al.[34] proposed the XDebloat which can help developers remove unwanted features and transform a bloated app into an instant app or an app bundle. Jiang et al.[23] studied bloated Android apps, and they built a prototype called RedDroid to identify the bloated app and remove dead code in apps.

6 CONCLUSION

In this study, we compare and analyze the differences between full apps and their corresponding lite versions. Our analysis includes a thorough examination of resource similarity, code similarity, permission consistency, privacy policy consistency, resource consistency, performance comparison, and feature practices in lite apps. The results of our study indicate that many existing lite apps do not achieve their intended goals, such as being smaller in size, faster performance, and consuming less data. Additionally, our study highlights the potential security risks that lite apps may pose. To assist app developers in building effective lite apps and debloating tools, we compare and discuss the functionalities in both lite and full apps.

7 DATA AVAILABILITY

The experimental results, raw data, and apps are available at: https://sites.google.com/view/liteapps/home.

REFERENCES

[1] Domenico Amalfitano, Anna Rita Fasolino, Porfirio Tramontana, Salvatore De Carmine, and Atif M. Memon. 2012. Using GUI ripping for automated testing of Android applications. In *Proceedings of ASE*. 258–261.

[2] Le An, Ons Mlouki, Foutse Khomh, and Giuliano Antoniol. 2017. Stack overflow: A code laundering platform?. In *2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 283–293.

[3] Android. 2022. AAPT. https://developer.android.com/studio/command-line/aapt2.

[4] Android. 2022. Coroutines on Android. https://developer.android.com/kotlin/coroutines.

[5] Appbrain. 2022. AppBrain. https://www.appbrain.com/.

[6] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps. In *Proceedings of PLDI*. 259–269.

[7] Kathy Wain Yee Au, Yi Fan Zhou, Zhen Huang, and David Lie. 2012. Pscout: analyzing the android permission specification. In *Proceedings of the 2012 ACM conference on Computer and communications security*. 217–228.

[8] Tanzirul Azim and Iulian Neamtiu. 2013. Targeted and Depth-First Exploration for Systematic Testing of Android Apps. *SIGPLAN Not.* 48, 10 (2013), 641–660.

[9] Suparna Bhattacharya, Kanchi Gopinath, and Mangala Gowri Nanda. 2013. Combining Concern Input with Program Analysis for Bloat Detection. In *Proceedings of OOPSLA*. Association for Computing Machinery, 745–764.

[10] Yinzhi Cao, Yanick Fratantonio, Antonio Bianchi, Manuel Egele, Christopher Kruegel, Giovanni Vigna, and Yan Chen. 2015. EdgeMiner: Automatically Detecting Implicit Control Flow Transitions through the Android Framework.. In *NDSS*.

[11] Diffuse. 2022. Diffuse is a tool for diffing APKs, AABs, AARs, and JARs. https://github.com/JakeWharton/diffuse.

[12] Haroon Elahi, Guojun Wang, and Jianer Chen. 2020. Pleasure or pain? An evaluation of the costs and utilities of bloatware applications in android smartphones. *Journal of Network and Computer Applications* 157 (2020), 102578.

[13] Felix Fischer, Konstantin Böttinger, Huang Xiao, Christian Stransky, Yasemin Acar, Michael Backes, and Sascha Fahl. 2017. Stack overflow considered harmful? the impact of copy&paste on android application security. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 121–136.

[14] Flurry. 2022. Flurry: Mobile App Analytics Platform for Android and iOS. https://www.flurry.com/.

[15] Tatiana Frenklach, Dvir Cohen, Asaf Shabtai, and Rami Puzis. 2021. Android malware detection via an app similarity graph. *Computers & Security* 109 (2021), 102386.

[16] Mohammad Gharehyazie, Baishakhi Ray, Mehdi Keshani, Masoumeh Soleimani Zavosht, Abbas Heydarnoori, and Vladimir Filkov. 2019. Cross-project code clones in github. *Empirical Software Engineering* 24, 3 (2019), 1538–1573.

[17] Hugo Gonzalez, Natalia Stakhanova, and Ali A Ghorbani. 2014. Droidkin: Lightweight detection of android apps similarity. In *International Conference on Security and Privacy in Communication Networks*. Springer, 436–453.

[18] Google. 2022. App Bundle. https://developer.android.com/platform/technology/app-bundle/.

[19] Google. 2022. Google Play. https://play.google.com/store/apps.

[20] GoogleAnalystics. 2022. GoogleAnalystics. https://analytics.google.com/analytics/web/.

[21] Wunan Guo, Liwei Shen, Ting Su, Xin Peng, and Weiyang Xie. 2020. Improving Automated GUI Exploration of Android Apps via Static Dependency Analysis. In *Proceedings of ICSME*. 557–568.

[22] Jianjun Huang, Yousra Aafer, David Perry, Xiangyu Zhang, and Chen Tian. 2017. UI driven Android application reduction. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 286–296.

[23] Yufei Jiang, Qinkun Bao, Shuai Wang, Xiao Liu, and Dinghao Wu. 2018. RedDroid: Android Application Redundancy Customization Based on Static Analysis. In *Proceedings of ISSRE*. 189–199.

[24] Ding Li, Yingjun Lyu, Mian Wan, and William GJ Halfond. 2015. String analysis for Java and Android applications. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 661–672.

[25] Li Li, Tegawendé F Bissyandé, and Jacques Klein. 2017. Simidroid: Identifying and explaining similarities in android apps. In *2017 IEEE Trustcom/BigDataSE/ICESS*. 136–143.

[26] Riyadh Mahmood, Nariman Mirzaei, and Sam Malek. 2014. EvoDroid: Segmented Evolutionary Testing of Android Apps. In *Proceedings of ESEC/FSE*. 599–609.

[27] Ke Mao, Mark Harman, and Yue Jia. 2016. Sapienz: Multi-Objective Automated Testing for Android Applications. In *Proceedings of ISSTA*. 94–105.

[28] Patrick McDaniel. 2012. Bloatware comes to the smartphone. *IEEE Security & Privacy* 10, 4 (2012), 85–87.

[29] OKHTTP3. 2022. OkHttp. https://square.github.io/okhttp/.

[30] Zhengyang Qu, Vaibhav Rastogi, Xinyi Zhang, Yan Chen, Tiantian Zhu, and Zhong Chen. 2014. Autocog: Measuring the description-to-permission fidelity in android applications. In *Proceedings of CCS*. 1354–1365.

[31] Atanas Rountev and Dacong Yan. 2014. Static Reference Analysis for GUI Objects in Android Software. In *Proceedings of CGO*. 143–153.

[32] Soot. 2022. Soot - A Java optimization framework. https://github.com/soot-oss/soot.

[33] Ting Su, Guozhu Meng, Yuting Chen, Ke Wu, Weiming Yang, Yao Yao, Geguang Pu, Yang Liu, and Zhendong Su. 2017. Guided, Stochastic Model-Based GUI Testing of Android Apps. In *Proceedings of ESEC/FSE*. 245–256.

[34] Yutian Tang, Hao Zhou, Xiapu Luo, Ting Chen, Haoyu Wang, Zhou Xu, and Yan Cai. 2021. XDebloat: Towards Automated Feature-Oriented App Debloating. *IEEE Transactions on Software Engineering* (2021), 1–19.

[35] Xiaoyin Wang, Xue Qin, Mitra Bokaei Hosseini, Rocky Slavin, Travis D. Breaux, and Jianwei Niu. 2018. GUILeak: Tracing Privacy Policy Claims on User Input Data for Android Applications. In *Proceedings of ICSE*. 37–47.

[36] Shengqian Yang, Haowei Wu, Hailong Zhang, Yan Wang, Chandrasekar Swaminathan, Dacong Yan, and Atanas Rountev. 2018. Static window transition graphs for Android. *Automated Software Engineering* 25, 4 (2018), 833–873.

[37] Shengqian Yang, Dacong Yan, Haowei Wu, Yan Wang, and Atanas Rountev. 2015. Static Control-Flow Analysis of User-Driven Callbacks in Android Applications. In *Proceedings of ICSE*. 89–99.

[38] Wu Zhou, Yajin Zhou, Xuxian Jiang, and Peng Ning. 2012. Detecting repackaged smartphone applications in third-party android marketplaces. In *Proceedings of the second ACM conference on Data and Application Security and Privacy*. 317–326.